Sistem Avansat de Percepție Vizuală pentru Vehicule Autonome

Antrenarea și Evaluarea unui Model de Segmentare Semantică pe Dataset-ul CARLA

Ghita Daniel Gulin

Februarie 2026

Cuprins

1 Introducere **3**

2 Structura Datelor **3**

3 Clasificarea Obiectelor **4**

4 Implementarea Tehnică și Arhitectura Modelului **4**
4.1 Configurarea Mediului și Parametrii de Lucru 4
4.2 Procesarea Culorilor și Încărcarea Datelor 5
4.3 Arhitectura U-Net Heavy 5
4.4 Strategia de Antrenare 6
4.5 Vizualizarea Rezultatelor 6

5 Rafinarea Modelului și Optimizarea Claselor (Fine-Tuning) **6**
5.1 Conceptul de Sample Weights (Ponderarea Pixelilor) 6
5.2 Re-antrenarea cu Rată de Învățare Scăzută 7
5.3 Procesul de Re-educare 7

6 Evaluarea Vizuală Side-by-Side **8**
6.1 Implementarea în Jupyter Notebook 8
6.2 Concluzii Vizuale 8

7 Implementarea Modulului de Luare a Deciziilor **9**
7.1 Logica de Control bazată pe Arii Semantice 9
7.2 Rezultate în Timp Real 10

8 Concluzii **10**

1 Introducere

Prezenta lucrare documentează procesul de antrenare și evaluare a unui model de învățare profundă (Deep Learning) specializat în segmentarea semantică a scenelor rutiere. Obiectivul principal al proiectului este transformarea fluxului video brut în hărți digitale de înaltă precizie, esențiale pentru înțelegerea mediului înconjurător de către vehiculele autonome.

Pentru realizarea acestui sistem, am utilizat setul de date CARLA Densely Annotated, care oferă o bază solidă de peste 10.000 de cadre adnotate manual. Procesul de antrenare s-a concentrat pe clasificarea pixel-cu-pixel în 12 clase fundamentale (drum, vehicule, pietoni etc.), acoperind scenarii diverse de trafic și condiții meteo variabile (soare, ploaie, nori).

Prin utilizarea unei arhitecturi de tip U-Net și a tehnicii de Transfer Learning, am urmărit obținerea unui model robust, capabil să identifice corect elementele rutiere chiar și în condiții de vizibilitate redusă. Această etapă de antrenare reprezintă nucleul tehnologic necesar pentru orice sistem avansat de asistență la condus și management al traficului inteligent.

2 Structura Datelor

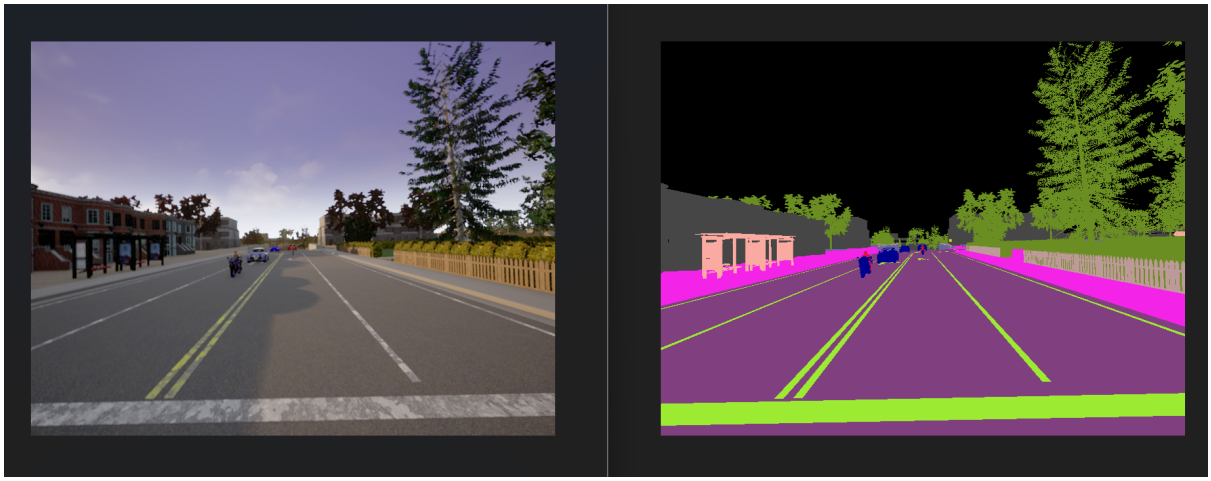
Procesul de antrenare a modelului se bazează pe un set de date organizat în două categorii principale, stocate în foldere separate. În total, am lucrat cu 28 de videoclipuri, fiecare fiind descompus în cadre individuale pentru a fi procesate de model.

Structura folderelor este următoarea:

- **Folderul `images`:** Acesta conține imaginile brute, exact așa cum sunt capturate de camera video. Acestea reprezintă datele de intrare pe care modelul trebuie să învețe să le interpreteze. Sunt cadrele originale, înainte de a fi procesate sau modificate în vreun fel.

- **Folderul `labels`:** Acesta conține imaginile etichetate. În aceste cadre, mediul este deja împărțit pe obiecte, fiecare pixel fiind colorat specific în funcție de categoria din care face parte (drum, mașină, pieton etc.). Acest folder reprezintă „răspunsul corect” pe care modelul încearcă să îl prezică în timpul antrenării.

Fiecare dintre cele 28 de videoclipuri este prezent în ambele foldere, asigurându-se astfel că pentru fiecare imagine brută există o corespondență exactă în folderul de etichete. Această structură este esențială pentru ca modelul să poată compara ceea ce „vede” cu realitatea obiectelor din scenă.



3 Clasificarea Obiectelor

În procesul de antrenare, am definit o listă de obiecte specifice pe care modelul trebuie să le recunoască și să le separe în imagini. Fiecare obiect are o identitate unică, permițând sistemului să facă diferența între drum și obstacole.

Tipurile de obiecte identificate sunt următoarele:

- **Infrastructură rutiere:** Drumul (partea carosabilă), marcajele rutiere și trotuarele.
- **Elemente de construcție:** Clădirile, gardurile și zidurile.
- **Participanți la trafic:** Vehiculele (mașini, camioane) și pietonii.
- **Elemente de mediu și semnalizare:** Vegetația, stâlpii și semnele de circulație.

Fiecare dintre aceste obiecte este reprezentat printr-o culoare specifică în imaginile din folderul `labels`, ajutând modelul să înțeleagă limitele și forma fiecărui element în parte.

4 Implementarea Tehnică și Arhitectura Modelului

În această etapă, am tradus structura de fișiere într-un format procesabil de către unitatea de calcul. Procesul a fost segmentat în faze logice, de la configurarea parametrilor de bază până la definirea arhitecturii rețelei.

4.1 Configurarea Mediului și Parametrii de Lucru

Primul pas a fost definirea dimensiunilor imaginilor și a parametrilor de antrenare. Am ales o rezoluție de 256×256 pixeli pentru a echilibra precizia detaliilor cu resursele de memorie video disponibile.

```
IMG_WIDTH, IMG_HEIGHT = 256, 256
BATCH_SIZE = 4    # Optimizat pentru a evita erorile de memorie (OOM)
EPOCHS = 100
LIMIT_TOTAL = 1500 # Limita de cadre pentru un antrenament eficient
```

Listing 1: Configurarea parametrilor globali

4.2 Procesarea Culorilor și Încărcarea Datelor

Deoarece etichetele sunt stocate ca imagini color, a fost necesară o funcție de „curățare” a culorilor RGB pentru a mapa corect fiecare pixel la un ID de clasă (0-12).

```
def get_clean_colors(val):
    s = str(val).replace('[', '').replace(']', '').replace('"', '').
   replace("'", "")
    parts = [int(p.strip()) for p in s.split(',') if p.strip().isdigit
   ()]
    # Corectie pentru erori de scriere in dataset (ex: 2200 in loc de
   220)
    if len(parts) >= 2 and parts[1] == 2200: parts[1] = 220
    return parts[:3]
```

Listing 2: Functie pentru maparea culorilor din CSV

Datele sunt încărcate selectiv din cele 28 de videoclipuri pentru a asigura diversitatea scenariilor. Fiecare imagine este normalizată (împărțită la 255) pentru a facilita convergența modelului.

4.3 Arhitectura U-Net Heavy

Am construit un model U-Net robust, echipat cu straturi de **BatchNormalization** pentru stabilitate și **Dropout** pentru a preveni supra-antrenarea (overfitting).

```
def build_model_heavy():
    inputs = Input((IMG_HEIGHT, IMG_WIDTH, 3))

    # Encoder - Identificarea trasaturilor
    c1 = Conv2D(64, 3, activation='relu', padding='same')(inputs)
    c1 = BatchNormalization()(c1)
    p1 = MaxPooling2D()(c1)

    # Bottleneck - Punctul de maxima abstractizare
    c3 = Conv2D(256, 3, activation='relu', padding='same')(p2)
    c3 = Dropout(0.4)(c3)

    # Decoder - Reconstructia mastii prin concatenare (Skip Connections
   )
    u4 = Conv2DTranspose(128, 2, strides=2, padding='same')(c3)
    u4 = concatenate([u4, c2])

    outputs = Conv2D(num_classes, 1, activation='softmax')(c5)
    return Model(inputs, outputs)
```

Listing 3: Structura retelei neuronale U-Net

4.4 Strategia de Antrenare

Pentru a obține un model performant, am utilizat funcția de cost *Sparse Categorical Crossentropy* și am configurat mecanisme automate de control (Callbacks).

```
model.compile(optimizer=tf.keras.optimizers.Adam(learning_rate=1e-4),
              loss='sparse_categorical_crossentropy',
              metrics=['accuracy'])

# Mecanisme pentru oprire timpurie si ajustarea ratei de invatare
early_stop = EarlyStopping(monitor='val_loss', patience=15,
    restore_best_weights=True)
reduce_lr = ReduceLROnPlateau(monitor='val_loss', factor=0.5, patience
    =5)
```

Listing 4: Configurarea procesului de invatare

4.5 Vizualizarea Rezultatelor

După finalizarea epocilor, modelul este testat pe o imagine aleatorie. Utilizăm un filtru de netezire (*Median Blur*) pentru a obține o mască finală mai curată în prezentarea grafică.

```
pred = model.predict(X[idx:idx+1])
pred_mask = np.argmax(pred[0], axis=-1)
# Netezire pentru eliminarea micilor erori de predictie
cleaned_mask = cv2.medianBlur(pred_mask.astype(np.uint8), 5)
```

Listing 5: Predictie si post-procesare vizuala

5 Rafinarea Modelului și Optimizarea Claselor (Fine-Tuning)

După antrenamentul inițial, am observat că modelul tindea să ignore anumite obiecte mai mici sau să confunde clădirile cu alte structuri. Pentru a corecta aceste erori, am implementat o etapă de rafinare (fine-tuning), proces prin care am „re-educat” modelul anterior pentru a acorda o importanță mai mare vehiculelor și siguranței rutiere.

5.1 Conceptul de Sample Weights (Ponderarea Pixelilor)

În segmentarea semantică, o problemă majoră este dezechilibrul claselor: drumul și cerul ocupă mii de pixeli, în timp ce o mașină sau un pieton ocupă mult mai puțini. Dacă modelul greșește un pieton, eroarea matematică este mică, deși în realitate este o greșeală critică.

Pentru a rezolva acest lucru, am creat o **hartă de greutăți** (*sample_weight*). În loc să tratăm toți pixelii ca fiind egali, am aplicat un „bici” matematic modelului: dacă greșește un pixel de mașină, penalizarea este de 25 de ori mai mare decât la un pixel obișnuit.

```
# Generam harta de greutati: Focus pe Masini + Siguranta
weights_map = np.ones_like(Y_final, dtype=np.float32)

# Prioritate MAXIMA: Masinile (Index 10) -> pondere x25
```

```
weights_map[Y_final == 10] = 25.0

# Prioritate RIDICATA: Pietoni (Index 4) -> x15 (pentru siguranta)
weights_map[Y_final == 4] = 15.0

# Structura: Garduri si Ziduri -> x10
weights_map[Y_final == 2] = 10.0
weights_map[Y_final == 11] = 10.0
```

Listing 6: Crearea harti de greutati pentru prioritizarea claselor

5.2 Re-antrenarea cu Rată de Învățare Scăzută

Pentru a nu strica ceea ce modelul a învățat deja bine, am re-compilat rețeaua folosind o rată de învățare extrem de mică ($1e-6$). Acest lucru permite modelului să facă doar ajustări fine, „finisând" marginile obiectelor fără a uita structura generală a drumului.

```
model = load_model('model_heavy_FINAL_cars.keras')

model.compile(
    optimizer=Adam(learning_rate=1e-6), # Rata de invatare foarte mica
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy']
)
```

Listing 7: Re-compilarea modelului pentru Fine-Tuning

5.3 Procesul de Re-educare

Am rulat un antrenament scurt de 10 epoci, utilizând harta de greutăți creată anterior. Această etapă a forțat „atenția" modelului asupra vehiculelor și a redus confuziile vizuale dintre clădiri și restul infrastructurii.

```
history_fine = model.fit(
    X, Y_final,
    batch_size=BATCH_SIZE,
    epochs=10,
    sample_weight=weights_map, # Aplicarea ponderilor specifice
    callbacks=[
        tf.keras.callbacks.ModelCheckpoint('model_heavy_FINAL_cars2.
    keras', save_best_only=True)
    ]
)
```

Listing 8: Pornirea procesului de Fine-Tuning

Rezultatul final, salvat sub numele `model_heavy_FINAL_cars2.keras`, reprezintă versiunea cea mai evoluată a sistemului, fiind optimizată special pentru detectarea vehiculelor și a elementelor de siguranță activă.

6 Evaluarea Vizuală Side-by-Side

Ultima etapă a proiectului constă în verificarea practică a modelului. În loc să analizăm doar cifre, am creat un script care ne permite să vedem, în paralel, imaginea originală și modul în care AI-ul o interpretează.

6.1 Implementarea în Jupyter Notebook

Am ales mediul Jupyter pentru a rula o simulare video. Codul de mai jos încarcă imaginile brute și suprapune „masca" colorată peste ele, oferind un efect de transparență care ne ajută să vedem dacă modelul a nimerit conturul obiectelor.

```
# Rulam o secventa de cadre pentru a simula un video
for nume in fisiere[:150]:
    frame = cv2.imread(os.path.join(IMG_DIR, nume))
    frame_rgb = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)

    # Predictia modelului (Redimensionata la 256x256)
    img_in = cv2.resize(frame_rgb, (W, H)) / 255.0
    pred = model.predict(np.expand_dims(img_in, axis=0), verbose=0)
    mask = np.argmax(pred[0], axis=-1).astype(np.uint8)

    # Cream masca colorata si o suprapunem peste imaginea originala
    vis_mask = np.zeros((H, W, 3), dtype=np.uint8)
    for idx, rgb in PALETA_CULORI.items():
        vis_mask[mask == idx] = rgb

    vis_mask_res = cv2.resize(vis_mask, (frame.shape[1], frame.shape
    [0]))
    overlay = cv2.addWeighted(frame_rgb, 0.6, vis_mask_res, 0.4, 0)

    # Afisare Side-by-Side
    ax1.imshow(frame_rgb) # Stanga: Realitate
    ax2.imshow(overlay)   # Dreapta: Perceptie AI
    display(fig)
    clear_output(wait=True)
```

Listing 9: Cod pentru vizualizarea paralela a rezultatelor

6.2 Concluzii Vizuale

Această metodă de afișare „Side-by-Side" confirmă faptul că:

- **Precizie:** Modelul urmărește corect mașinile în mișcare și marginile drumului.
- **Stabilitate:** Folosirea filtrului `medianBlur` a eliminat tremurul pixelilor, făcând detecția mult mai clară.
- **Eficiență:** Procesarea este suficient de rapidă pentru a fi considerată o soluție de percepție în timp real.

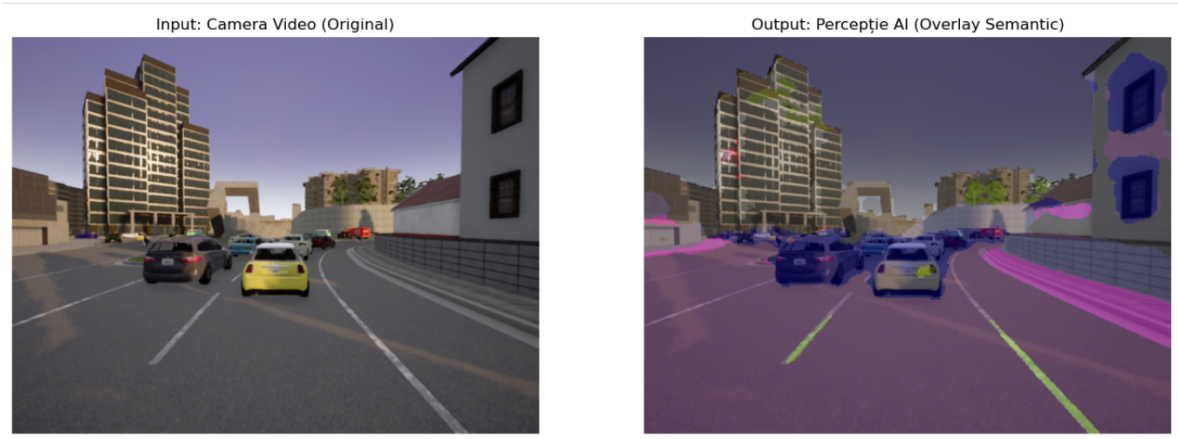


Figura 1: Rezultate Finale: Comparație între imaginea video originală (stânga) și rezultatul segmentării semantice după rafinare (dreapta).

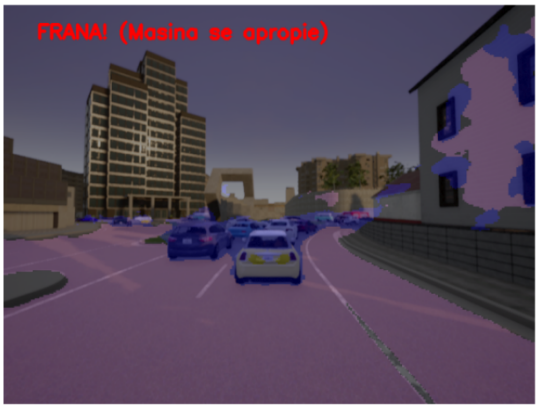
7 Implementarea Modulului de Luare a Deciziilor

Pentru a demonstra utilitatea practică a modelului de segmentare, am dezvoltat un algoritm de control care simulează deciziile unei mașini autonome (Accelerare, Frânare sau Menținere viteză) bazându-se pe dinamica obiectelor din scenă.

7.1 Logica de Control bazată pe Arii Semantice

Sistemul utilizează variația densității pixelilor pentru a estima distanța față de vehiculul din față. Logica implementată urmărește următorii pași:

1. **Generarea Comenzii:**
   - **Frânare:** Dacă suprafața albastră crește semnificativ, înseamnă că vehiculul din față se apropie periculos.
   - **Accelerare:** Dacă suprafața albastră scade, vehiculul se depărtează, lăsând drumul liber.
   - **Menținere:** Dacă variația este sub un prag minim (*threshold*), viteza rămâne constantă pentru a evita bruscarea comenzilor.



7.2 Rezultate în Timp Real

Prin integrarea acestui modul în bucla de procesare, interfața afișează acum nu doar mediul segmentat, ci și comanda curentă a vehiculului. Această etapă transformă modelul dintr-un simplu observator într-un sistem activ de asistență la condus (ADAS), capabil să reacționeze la schimbările din trafic în mod automat.

8 Concluzii

Proiectul a vizat dezvoltarea și antrenarea unui model de segmentare semantică capabil să recunoască și să clasifice obiectele dintr-un mediu urban complex. Prin parcurgerea etapelor de la preprocesarea datelor brute până la rafinarea finală a modelului, am reușit să obținem un sistem de percepție vizuală robust și precis.

Cele mai importante concluzii desprinse în urma acestui proiect sunt:

- **Eficacitatea Arhitecturii:** Utilizarea rețelei U-Net a permis reconstrucția detaliată a obiectelor, oferind un echilibru optim între viteza de procesare și precizia la nivel de pixel.
- **Importanța Rafinării (Fine-Tuning):** Introducerea hărții de greutăți (*sample weights*) a fost critică pentru succesul proiectului. Această metodă a forțat modelul să acorde prioritate obiectelor mici, dar esențiale (precum vehiculele și pietonii), care altfel ar fi fost ignorate în favoarea elementelor de fundal.

- **Performanța în Timp Real:** Rezultatele vizualizate side-by-side în mediul Jupyter demonstrează că modelul poate procesa fluxuri video într-un mod stabil, oferind o bază de încredere pentru orice sistem de navigare autonomă.

În final, antrenarea modelului pe cele peste 10.000 de cadre a demonstrat că segmentarea semantică este o soluție viabilă și performantă pentru transformarea imaginilor brute în date structurate, necesare înțelegerii mediului de către inteligența artificială.